

LOOPS

Types of loops

for loop: Executes code with a counter.

while loop: Executes while condition is true.

do-while: Executes first, then checks condition.

for-in: Iterates over object keys.

for-of: Iterates over iterable values (array, string).

```
for(let i=0; i<5; i++){  
  console.log(i);  
}
```

```
let j = 0;  
while(j<5){  
  console.log(j);  
  j++;  
}
```

```
let k = 0;  
do {  
  console.log(k);  
  k++;  
} while(k<5);
```

```
let obj = {a:1,b:2};  
for(let key in obj){  
  console.log(key, obj[key]);  
}
```

```
}
```

```
let arr = [10,20,30];  
for(let value of arr){  
  console.log(value);  
}
```

Interview Questions

Q. Difference between for-in and for-of.

for-in

- Iterates over **enumerable property keys** of an object.
- Works on **objects** and **arrays**, but for arrays it gives **indices**, not values.

for-of

- Iterates over **iterable values** (arrays, strings, Maps, Sets, etc.).
- Gives **values directly**, not keys.
- Doesn't work on plain objects (objects are not iterable by default).

A **plain object** is an object that is **directly created by the Object constructor** or with **object literal syntax ({})** and **does not have a custom prototype or class**.

// Plain object examples

```
const obj1 = {};           // Object literal
```

```
const obj2 = new Object(); // Using Object constructor
```

```
// non plain object example

class Person {
  constructor(name) {
    this.name = name;
  }
}

const p = new Person("Alice"); // Not a plain object

const arr = [1, 2, 3]; // Array, not a plain object
const map = new Map();
```

2. When to use while vs do-while.

Use while → “Run only if the condition is true.”

Use do-while → “Run at least once, then check the condition.”

3. How to prevent infinite loops.

“Infinite loops happen when a loop’s exit condition is never met. To prevent them, I make sure the loop control variable or condition is updated correctly, use safeguards like maximum iterations or break statements for dynamic conditions, and prefer structured loops like for when possible. Essentially, I ensure every loop has a clear and reachable exit path

4. Can you use break and continue inside loops? Explain.

1 break

- **Purpose:** Immediately **terminates the current loop**, skipping any remaining iterations.

2 continue

- **Purpose: Skips the current iteration** and moves to the next iteration of the loop.

“Yes, both break and continue can be used inside loops. break exits the loop entirely, while continue skips the current iteration and moves to the next. They help control loop flow efficiently.”